EXPERIMENT NO. 05 (Software) – Socket Programming



Name: Dibyanshu Shekhar Dey
Roll Number: 25EC64R23

Design and Simulation Lab: Communication Networks
Department of Electronics & Electrical Communication Engineering
Indian Institute of Technology Kharagpur

AIM:
Demo 1: Demonstrate TCP-based client–server communication with multiple clients using socket programming.
Task

- Implement a TCP server listening on port 5000
- Three clients connect to the server
- Each client sends a distinct message:
  - Client 1: `"Message from Client 1"`
  - Client 2: `"Message from Client 2"`
  - Client 3: `"Message from Client 3"`
- The server echoes the received message back to the corresponding client

Demo 2: TCP vs UDP Performance Comparison
Objective Evaluate the performance differences between TCP and UDP communication using socket programming.
Task

- Implement TCP and UDP echo servers listening on port 5000
- A client sends **1000 messages**: `"Performance Test Message"`
- The server echoes each message back to the client

Measurements

- Compute the **average Round Trip Time (RTT)**
- Measure **packet loss** (messages without response)

Hint: Use `struct timeval start, end` with `gettimeofday()`

$$RTT\ (ms) = (end.tv_sec - start.tv_sec) \times 1000 + \frac{(end.tv_usec - start.tv_usec)}{1000}$$

Demo 3: Server Scheduling
Objective : Study how scheduling policies affect server performance when multiple clients submit jobs concurrently.
Task

- Implement a TCP server listening on port **5000**.
- Launch at least **5 concurrent clients**. Each client submits at least **3 job requests**.
- Each job includes:
  - processing time: **1–5 seconds**
  - priority: **1–5** (higher value = higher priority)

- Implement both **FCFS** and **Priority Scheduling** at the server and compare their performance using the metrics **waiting time** and **response time**.

INTRODUCTION:
The TCP/IP model gives a layered architecture for communication between various applications that are running on different machines in a network. Communication as such in the newtrok is handled through transport layer protocols such as TCP and UDP, while the operating system (OS) offers an interface called a **socket** enabling applications to access these services. Socket programming allows applications to send and receive data over a network.

In distributed environments, multiple clients and servers interact, such as users accessing services or applications exchanging data. Since processes on different machines cannot directly share memory or use standard inter-process communication, network-based communication using sockets becomes essential.

THEORY:
Transmission Control Protocol (TCP) – Summary
TCP (Transmission Control Protocol) is a transport layer protocol that enables reliable communication over a network, therefore ensuring that the data is delivered accurately, in order, and without loss, even in unreliable network conditions [1].

- Operates at the **Transport Layer** of the OSI model and is a core part of the TCP/IP suite.
- Establishes a logical connection between sender and receiver before data transmission.
- Guarantees ordered and reliable delivery using acknowledgements and sequence numbers.
- Detects errors using check sums and retransmits lost or corrupted packets.
- Implementation of flow and congestion control to regulate data transmission efficiently.

User Datagram Protocol (UDP) UDP (User Datagram Protocol) is a transport layer protocol that provides fast, connectionless, and lightweight communication between processes. It does not guarantee reliable delivery, ordering, or error correction, making it suitable for real-time and latency-sensitive applications [2].
Key Characteristics

- Connectionless communication (no setup required)
- Low latency and minimal overhead
- No guarantee of delivery, order, or error recovery
- Suitable for time-sensitive applications

Applications of UDP

- **DNS:** Fast query-response communication
- **DHCP:** Dynamic IP address allocation
- **VoIP:** Real-time voice communication

- **RIP:** Routing updates between routers
- **NTP:** Time synchronization across networks

TCP/IP Architecture Summary

TCP and IP are two fundamental protocols: Transmission Control Protocol (TCP) and Internet Protocol (IP). Together, they form the TCP/IP architecture, which is designed for communication across large networks composed of interconnected segments.

The TCP/IP model is organized as a layered stack, where each layer performs specific functions and provides services to the layer above it. The architecture consists of four layers:

- **Application Layer:** Provides network services to user applications (e.g., HTTP, FTP, Telnet).
- **Transport Layer:** Handles end-to-end communication using protocols such as TCP and UDP.
- **Internet Layer:** Responsible for logical addressing and routing using IP.
- **Network Access Layer:** Deals with hardware, device drivers, and physical network transmission.

Each layer depends on the services of the lower layers while abstracting implementation details from the upper layers. This layered design enables efficient communication between user processes and underlying network mechanisms [3].

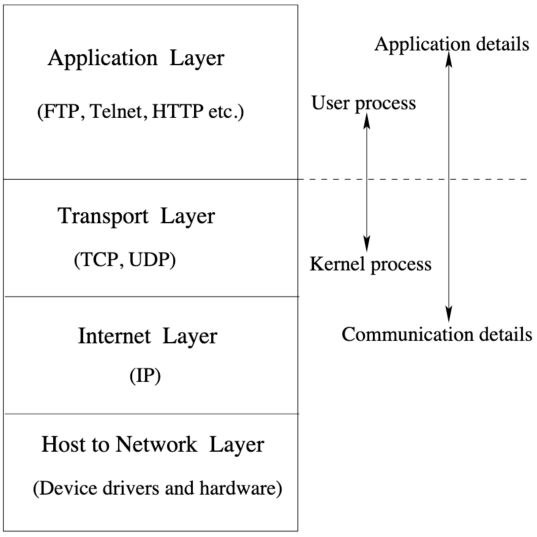


Figure 1: TCP/IP Architechture.[3]

Basic Socket Programming Functions
Socket programming is performed using system calls provided by the operating system. These functions allow applications to create communication endpoints and exchange data over a network [4].

Function	Purpose
`socket()`	Create a communication endpoint
`bind()`	Associate socket with an IP address and port
`listen()`	Enable server to accept connections
`accept()`	Accept a client connection
`connect()`	Initiate connection to server
`send()/recv()`	Exchange data between client and server
`close()`	Terminate the connection

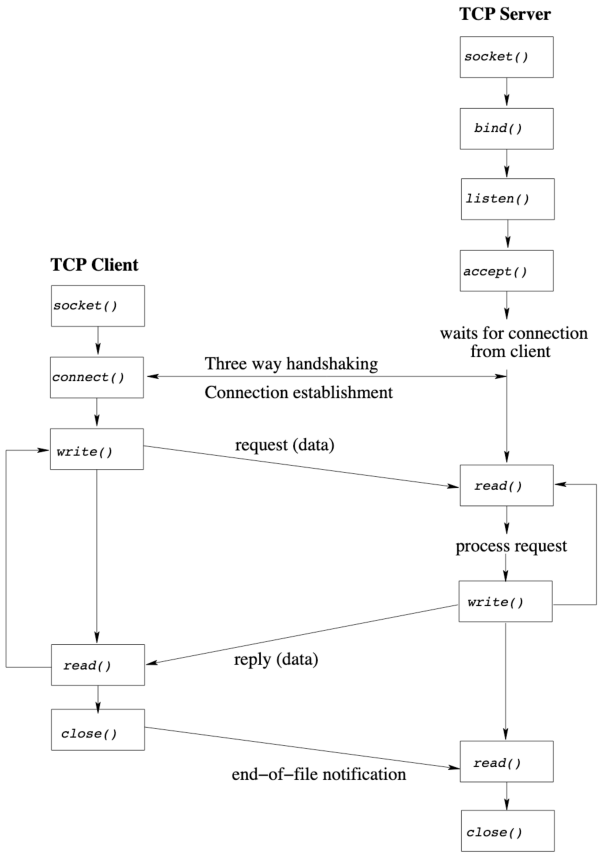


Figure 2: Socket Functions and their sequences for TCP/IP Client and server.[3]

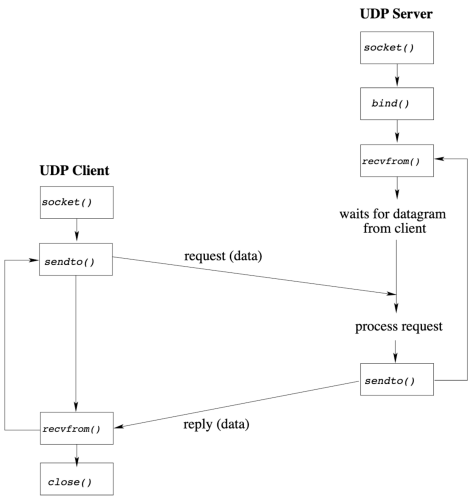


Figure 3: Socket Functions and their sequences for UDP Client and server.[3]

RESULTS:

Demo 1:

```
(base) dibyanshu@Dibyanshus-MacBook-Air Day_1 % gcc server.c
(base) dibyanshu@Dibyanshus-MacBook-Air Day_1 % ./a.out
Message from Client 1
Message from Client 2
Message from Client 3
```

Figure 4: Server Terminal Output after running server.c and running all the different client programs.

```
(base) dibyanshu@Dibyanshus-MacBook-Air Day_1 % gcc server.c
(base) dibyanshu@Dibyanshus-MacBook-Air Day_1 % ./a.out
Message from Client 1
Message from Client 2
Message from Client 3
```

Figure 5: Client Output after running client.c .

Demo 2:

```
(base) dibyanshu@Dibyanshus-MacBook-Air GPT_Demo_2 % gcc tcp_server.c
(base) dibyanshu@Dibyanshus-MacBook-Air GPT_Demo_2 % ./a.out
TCP Server listening on port 55000...
```

Figure 6: Server terminal output after running `tcp_server.c`

```
Performace Test Message 998
Performace Test Message 999
Performace Test Message 1000
TCP Average RTT = 0.036 ms
TCP Packet Loss = 0
```

Figure 7: Client Terminal output after running `tcp_client.c`

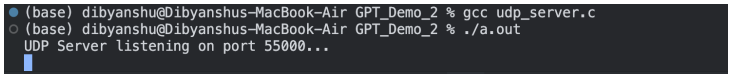


Figure 8: Client Terminal output after running `udp_server.c`

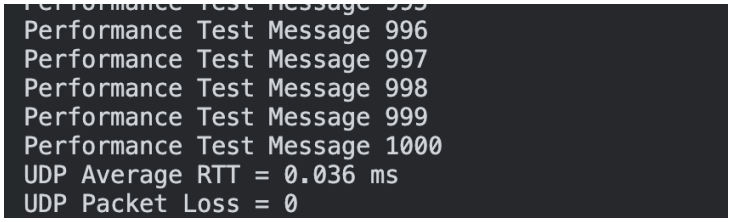


Figure 9: Client Terminal output after running `udp_client.c`

DISCUSSION:

What is a port? Do we need different ports in a multi-client architecture? An IP address uniquely identifies a host in a network, but it is not sufficient to identify specific applications that are running on that host. To distinguish between multiple applications, the transport layer uses port numbers. Each application is assigned a unique port number, allowing incoming packets to be delivered to the correct application process [3].

Based on the experiment done in the lab, we do not need different ports in a multi-client architecture. However we need to be careful about the ports. Suppose a server has been assigned a port, multiple client need to have the same port numbers so that a connection is established. In this sense, the answer of the question is valid.

Can we have the same IP address for different clients and still have a working multi-client architecture? How? Yes, multiple clients can share the same public IP address and still maintain a working multi-client architecture. This happens in server's end. Servers distinguish between clients sharing the same IP address by utilizing unique source ports, y creating a distinct 4-tuple (Source IP, Source Port, Destination IP, Destination Port) for each connection. This process enables the server to identify and treat each client's network connection as a separate, independent stream [5].

The RTT Time for TCP/IP and UDP was same with zero packet loss.
The Port number used in personal computer was different as this was reserved for some internal process. These sources were helpful in debugging it [6], [7].
Future Work: - Generate some packet losses and implementing Demo 3.

ACKNOWLEDGEMENT:

I would like to share my thanks to Prof. Amitalok J Budkuley for explaining the TCP/UDP Sockets. I would like to extend my thanks to TA MV Srivatsav for explaining the fundamentals of the experiment with great presentation and code, and helping throughout along with the problem description. I would like to thank my colleagues Mayank Singh, Saptarshi Bhattacharjee and Soumyajit Adak for collaborating in lab throughout the experiment.

References

[1] GeeksforGeeks, *What is transmission control protocol (tcp)?* https://www.geeksforgeeks.org/computer-networks/what-is-transmission-control-protocol-tcp/, 2025.

[2] GeeksforGeeks, *User datagram protocol (udp)*, https://www.geeksforgeeks.org/computer-networks/user-datagram-protocol-udp/, 2025.

[3] S. Das, *A short note on socket programming through c*, Department of Information Technology, Bengal Engineering and Science University, Shibpur, Assistant Professor, West Bengal, India - 711103.

[4] GeeksforGeeks, *Socket programming in c/c++*, https://www.geeksforgeeks.org/c/socket-programming-cc/, 2025.

[5] A. Agrawal, *How multiple simultaneous tcp connections work: A deep dive into the tcp 4-tuple & packet routing*, https://medium.com/@adityaagrawal399/how-multiple-simultaneous-tcp-connections-work-a-deep-dive-into-the-tcp-4-tuple-packet-routing-66a96153b0ae, 2024.

[6] Stack Overflow, *Socket programming: Setsockopt protocol not available*, 2019. [Online]. Available: https://stackoverflow.com/questions/58599070/socket-programming-setsockopt-protocol-not-available.

[7] nishio, *In macosx port:5000 is already in use by controlcenter*, 2023. [Online]. Available: https://github.com/nishio/external_brain_in_markdown/blob/main/pages/in%20MacOSX%20Port:5000%20is%20already%20in%20use%20by%20ControlCenter.md.

APPENDIX (CODE):

Demo 1
server.c

```c
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>
#define PORT 9000
int main(int argc, char const* argv[])
{
    int server_fd, new_socket;
    ssize_t valread;
    struct sockaddr_in address;
    int opt = 1;
    socklen_t addrlen = sizeof(address);
    char buffer[1024] = { 0 };

    // Creating socket file descriptor
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

```

```
    // Forcefully attaching socket to the port 8080
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR , &opt, sizeof(opt))
        ) {
        perror("setsockopt");
        exit(EXIT_FAILURE);
    }
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    // Forcefully attaching socket to the port 8080
    if (bind(server_fd, (struct sockaddr*)&address,
             sizeof(address))
        < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }
    if (listen(server_fd, 3) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }

    while(1){
        if ((new_socket
         = accept(server_fd, (struct sockaddr*)&address,
                  &addrlen))
        < 0) {
        perror("accept");
        exit(EXIT_FAILURE);
        }

        // subtract 1 for the null
        // terminator at the end
        valread = read(new_socket, buffer,
                       1024 - 1);
        printf("%s\n", buffer);
        send(new_socket, buffer, strlen(buffer), 0);

        // closing the connected socket
        close(new_socket);

    }
    // closing the listening socket
    close(server_fd);

    return 0;
}
```

client1.c

```
#include <arpa/inet.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>
#define PORT 9000

int main(int argc, char const* argv[])
{
```

```
    int status, valread, client_fd;
    struct sockaddr_in serv_addr;
    char* hello = "Message from Client 1";
    char buffer[1024] = { 0 };
    if ((client_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Convert IPv4 and IPv6 addresses from text to binary
    // form
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr)
        <= 0) {
        printf(
            "\nInvalid address/ Address not supported \n");
        return -1;
    }

    if ((status
         = connect(client_fd, (struct sockaddr*)&serv_addr,
                   sizeof(serv_addr)))
        < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }

    // subtract 1 for the null
    // terminator at the end
    send(client_fd, hello, strlen(hello), 0);

    valread = read(client_fd, buffer,
                   1024 - 1);
    printf("%s\n", buffer);

    // closing the connected socket
    close(client_fd);
    return 0;
}
```

Demo 2

tcp_server.c

```
// tcp_server.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 55000
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
```

```
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 3);

    printf("TCP Server listening on port %d...\n", PORT);

    new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t
        *)&addrlen);

    while (1) {
        int valread = read(new_socket, buffer, BUFFER_SIZE);
        if (valread <= 0) break;

        send(new_socket, buffer, valread, 0);
    }

    close(new_socket);
    close(server_fd);
    return 0;
}
```

tcp_client.c

```
// tcp_client.c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/time.h>

#define PORT 55000
#define NUM_MSG 1000

int main() {
    int sock;
    struct sockaddr_in server_addr;
    char *message = "Performance Test Message";
    char buffer[1024];

    struct timeval start, end;

    double total_rtt = 0;
    int received = 0;

    sock = socket(AF_INET, SOCK_STREAM, 0);

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);

    connect(sock, (struct sockaddr *)&server_addr, sizeof(server_addr));
```

```
    for (int i = 0; i < NUM_MSG; i++) {
        gettimeofday(&start, NULL);

        // Send message
        send(sock, message, strlen(message), 0);

        // Receive echo
        int n = read(sock, buffer, sizeof(buffer));

        gettimeofday(&end, NULL);

        if (n > 0) {
            buffer[n] = '\0';

            if (strcmp(buffer, message) == 0) {
                printf("Performace Test Message %d\n", i+1);
            } else {
                printf("Mismatch at message %d\n", i+1);
            }

            double rtt = (end.tv_sec - start.tv_sec) * 1000.0 +
                         (end.tv_usec - start.tv_usec) / 1000.0;

            total_rtt += rtt;
            received++;
        } else {
            printf("No response for message %d\n", i+1);
        }
    }

    printf("TCP Average RTT = %.3f ms\n", total_rtt / received);
    printf("TCP Packet Loss = %d\n", NUM_MSG - received);

    close(sock);
    return 0;
}
```

udp_server.c

```
// udp_server.c
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>

#define PORT 55000
#define BUFFER_SIZE 1024

int main() {
    int sockfd;
    char buffer[BUFFER_SIZE];
    struct sockaddr_in server_addr, client_addr;
    socklen_t len = sizeof(client_addr);

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
```

```
    server_addr.sin_port = htons(PORT);

    bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));

    printf("UDP Server listening on port %d...\n", PORT);

    while (1) {
        int n = recvfrom(sockfd, buffer, BUFFER_SIZE, 0,
                         (struct sockaddr *)&client_addr, &len);

        sendto(sockfd, buffer, n, 0,
               (struct sockaddr *)&client_addr, len);
    }

    close(sockfd);
    return 0;
}
```

udpp_client.c

```
// udp_client.c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/time.h>

#define PORT 55000
#define NUM_MSG 1000

int main() {
    int sockfd;
    struct sockaddr_in server_addr;
    char *message = "Performance Test Message";
    char buffer[1024];

    struct timeval start, end;
    double total_rtt = 0;
    int received = 0;

    socklen_t len = sizeof(server_addr);

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    // Set timeout (IMPORTANT for packet loss)
    struct timeval timeout;
    timeout.tv_sec = 1;
    timeout.tv_usec = 0;
    setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout));

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);

    for (int i = 0; i < NUM_MSG; i++) {
        gettimeofday(&start, NULL);

        sendto(sockfd, message, strlen(message), 0,
            (struct sockaddr *)&server_addr, len);
```

```
40
41          int n = recvfrom(sockfd, buffer, sizeof(buffer), 0,
42                           (struct sockaddr *)&server_addr, &len);
43
44          gettimeofday(&end, NULL);
45
46          if (n > 0) {
47              buffer[n] = '\0';  // null terminate
48
49              if (strcmp(buffer, message) == 0) {
50                  printf("Performance Test Message %d\n", i + 1);
51              } else {
52                  printf("Mismatch at message %d\n", i + 1);
53              }
54
55              double rtt = (end.tv_sec - start.tv_sec) * 1000.0 +
56                           (end.tv_usec - start.tv_usec) / 1000.0;
57
58              total_rtt += rtt;
59              received++;
60          } else {
61              // Packet loss case
62              printf("Packet lost at message %d\n", i + 1);
63          }
64      }
65
66      printf("UDP Average RTT = %.3f ms\n", total_rtt / received);
67      printf("UDP Packet Loss = %d\n", NUM_MSG - received);
68
69      close(sockfd);
70      return 0;
71  }
```